

Universidad Nacional Autónoma de México

Facultad de Ingeniería

Sistemas Operativos



Grupo 1

Ing. Angel Isidro Mercado

Tarea 6. Sistemas distribuidos

320304435 - Cabrera Rojas Oscar

Ciudad Universitaria, CDMX a 29 de octubre del 2026

Índice

1. Sistemas operativos de red y sistemas distribuidos **1**
- 1.1. Arquitectura y naturaleza de los sistemas operativos de red 1
- 1.2. Definición, diseño y principios del sistema operativo distribuido 2
- 1.3. Dimensiones de transparencia y características esenciales 2
- 1.4. Análisis comparativo: ventajas, desventajas y compromisos de diseño 3

2. Taxonomía de servicios y arquitecturas de servidores **4**
- 2.1. Modelos cliente-servidor y paradigmas de comunicación distribuida 5
- 2.2. Clasificación funcional de servidores en redes corporativas 6

3. Sistemas operativos orientados a servidores **8**
- 3.1. Arquitectura y especialización de sistemas operativos para servidor 8
- 3.2. Estudio técnico y comparativo de diez plataformas de servidor 9
  - 3.2.1. Microsoft Windows Server 9
  - 3.2.2. Red Hat Enterprise Linux (RHEL) 9
  - 3.2.3. Debian GNU/Linux 9
  - 3.2.4. Ubuntu Server 9
  - 3.2.5. SUSE Linux Enterprise Server (SLES) 9
  - 3.2.6. FreeBSD 10
  - 3.2.7. Oracle Solaris / illumos 10
  - 3.2.8. OpenBSD 10
  - 3.2.9. Rocky Linux / AlmaLinux 10
  - 3.2.10. VMware ESXi 10
  - 3.2.11. Matriz comparativa de sistemas operativos de servidor 10

4. Servidores de conectividad, enrutamiento y seguridad perimetral **11**
- 4.1. Servidores de traducción de direcciones de red (NAT) 12
- 4.2. Servidores de filtrado y cortafuegos (Firewall) 12
- 4.3. Servidores de configuración dinámica de direcciones (DHCP) 13
- 4.4. Servidores de intermediación y almacenamiento en caché (Proxy) 13

5. Servidores de autenticación, compartición, impresión y aplicaciones **14**
- 5.1. Servidores de autenticación y contabilidad de accesos (RADIUS) 15
- 5.2. Servidores de interoperabilidad y compartición de archivos (SAMBA) 15
- 5.3. Servidores de gestión y colas de impresión (CUPS) 16
- 5.4. Servidores de acceso remoto y pasarelas de conexión (RAS) 16
- 5.5. Servidores de infraestructura web integrada (pila LAMP) 16

6. Herramientas de entorno gráfico, desarrollo y capas de compatibilidad **18**
- 6.1. Gestores de composición de ventanas: Compiz 18
- 6.2. Entornos ligeros de desarrollo integrado: Geany 18
- 6.3. Capas de compatibilidad y traducción de API: Wine 19
- 6.4. Matriz comparativa de herramientas de software 20

7. Conclusiones personales **20**

Referencias **22**

1. Sistemas operativos de red y sistemas distribuidos

La evolución de las arquitecturas de cómputo transitó desde esquemas centralizados monoprocesador hacia plataformas donde la potencia de cálculo, la memoria y el almacenamiento residen en múltiples estaciones físicas interconectadas por redes de telecomunicaciones. La gestión de estos entornos plantea el reto de coordinar procesadores autónomos que operan concurrentemente sin compartir un bus de memoria física ni una señal de reloj global (Silberschatz et al., 2006, §1.10, p. 25). Esta transición dio origen a dos paradigmas estructurales: los sistemas operativos de red y los sistemas operativos distribuidos.

1.1. Arquitectura y naturaleza de los sistemas operativos de red

Un sistema operativo de red (*network operating system*, NOS) enlaza computadoras independientes a través de una red para facilitar el acceso a servicios y recursos compartidos (almacenamiento secundario, impresoras corporativas y servicios de cómputo remoto) conservando la autonomía de cada host (McHoes & Flynn, 2011, §10, p. 318). Su rasgo medular es la arquitectura desacoplada: cada nodo ejecuta su propio sistema operativo local de forma independiente, administrando sus componentes físicos mediante políticas y estructuras de control locales.

El NOS opera como una capa superpuesta al sistema anfitrión. Al emitirse una llamada al sistema para interactuar con un descriptor de archivo o dispositivo, el núcleo local determina si el destino es local o foráneo. Ante referencias remotas, el subsistema local redirige la solicitud a los módulos de cliente del NOS, los cuales encapsulan la instrucción en tramas de red bajo la pila TCP/IP hacia la estación de destino (McHoes & Flynn, 2011, §10, pp. 318–319). En el nodo receptor, el demonio servidor del NOS procesa el paquete y lo entrega a su propio núcleo para su resolución física, devolviendo el resultado por la red. Así, el NOS actúa como un facilitador de transporte y traducción entre núcleos autónomos, sin ejercer control global sobre el procesador, la memoria física o la planificación centralizada.

Los usuarios y procesos en un NOS son plenamente conscientes de la multiplicidad de equipos (Silberschatz et al., 2006, §16.2, pp. 559–561). Para consumir un recurso remoto, se requiere direccionar explícitamente el nodo servidor mediante herramientas y protocolos específicos:

1. **Inicio de sesión remoto (*remote login*):** Establece una terminal virtual interactiva mediante protocolos como SSH (*Secure Shell*) o Telnet, permitiendo ejecutar órdenes en el espacio de usuario remoto.
2. **Transferencia remota de archivos:** Moviliza ficheros entre sistemas de archivos dispares mediante protocolos de aplicación como FTP o SFTP, demandando comandos explícitos de transmisión.
3. **Sistemas de archivos de red:** Abstracciones como NFS o SMB/CIFS que montan jerarquías de directorios remotos en el árbol local del cliente, exigiendo especificar la ruta o dirección IP del servidor.

La principal ventaja del NOS es su modularidad y facilidad de implantación sobre sistemas operativos comerciales heterogéneos sin requerir modificaciones en sus núcleos (Deitel et al., 2004, §17.2, p. 787). Su limitación técnica reside en la falta de transparencia y en la delegación manual de la concurrencia y localización de recursos en el usuario.

1.2. Definición, diseño y principios del sistema operativo distribuido

Un sistema operativo distribuido (*distributed operating system*, DO/S) administra un conjunto de computadoras independientes presentando a usuarios y procesos una imagen de sistema único (*single-system image*, SSI) (Silberschatz et al., 2006, §1.10, p. 25, §16.2, p. 561). En un DO/S, la infraestructura física se unifica bajo una sola abstracción: el acceso a ficheros, procesadores y periféricos se ejecuta con las mismas primitivas y llamadas al sistema de un entorno monoprocesador, con total independencia de su localización física (Deitel et al., 2004, §17.2, pp. 787–788).

En esta arquitectura, los recursos no pertenecen a los nodos locales, sino al dominio global del DO/S (McHoes & Flynn, 2011, §10, p. 320). El núcleo distribuido implementa dos modelos estructurales:

- **DO/S basado en procesos:** Emplea núcleos cooperativos replicados en cada nodo que se comunican mediante paso de mensajes y canales IPC para prestar servicios de planificación, memoria y E/S mediante procesos servidores sincronizados (McHoes & Flynn, 2011, §10, pp. 323–324).
- **DO/S basado en objetos:** Modela el hardware y software como objetos discretos con interfaces formales que encapsulan su estado y se invocan de manera transparente a través de la red.

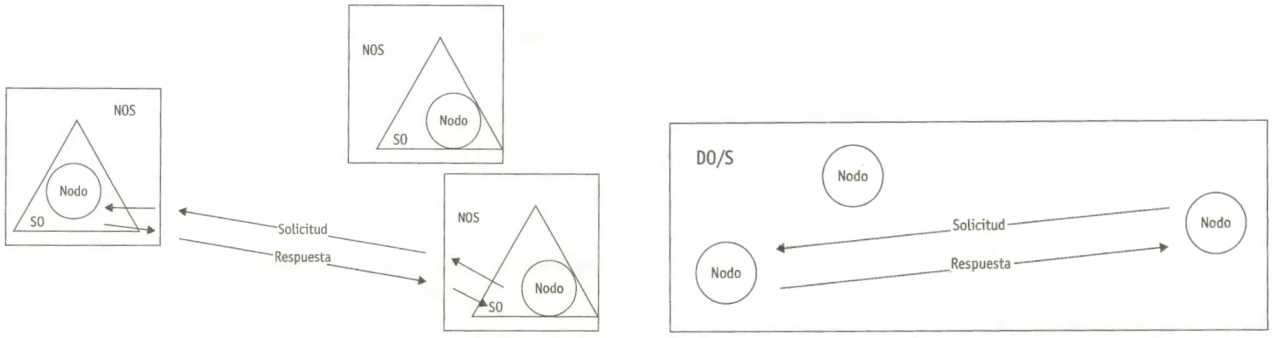


Imagen 1: Comparación estructural entre el modelo desacoplado de sistema operativo de red (NOS, izquierda) y el modelo integrado de sistema operativo distribuido (DO/S, derecha) (McHoes & Flynn, 2011, §10, pp. 319–320).

Como ilustra la Img. 1, el DO/S asume el control directo del hardware en cada nodo. En la administración de procesos, el planificador global distribuye la carga algorítmicamente mediante migración de procesos, transfiriendo de forma transparente el bloque de control del proceso (PCB), su contexto de ejecución y descriptores abiertos hacia los procesadores con menor carga (Silberschatz et al., 2006, §16.2, p. 563). En la gestión de memoria, implementa memoria compartida distribuida (*distributed shared memory*, DSM), donde los fallos de página sobre direcciones remotas activan transferencias de bloques en red sin intervención del hilo solicitante (McHoes & Flynn, 2011, §10, pp. 321–322).

1.3. Dimensiones de transparencia y características esenciales

El fin primordial del DO/S es la transparencia. El Modelo de Referencia para Procesamiento Distribuido Abierto (RM-ODP, ISO/IEC 10746) sistematiza ocho dimensiones de transparencia cardinales (Deitel et al., 2004, §17.2, pp. 786–787):

1. **Acceso (*access transparency*):** Oculta disparidades de protocolos y representaciones de datos (*endianness*, formatos de coma flotante) entre arquitecturas heterogéneas.
2. **Ubicación (*location transparency*):** Oculta la localización física de los recursos; las entidades se direccionan mediante nombres lógicos globales sin conocer el host anfitrión.
3. **Migración (*migration transparency*):** Permite transferir recursos o procesos entre nodos sin suspender la operación de los clientes asociados.
4. **Reubicación (*relocation transparency*):** Enmascara la relocalización de un componente mientras éste se encuentra siendo accedido concurrentemente por otros objetos.
5. **Replicación (*replication transparency*):** Mantiene copias redundantes para alta disponibilidad y desempeño, mostrándolas como una única instancia consistente.
6. **Concurrencia (*concurrency transparency*):** Permite el acceso simultáneo a recursos compartidos garantizando consistencia sin interferencias destructivas.
7. **Fallos (*failure transparency*):** Oculta la interrupción de nodos o enlaces de comunicación mediante puntos de control (*checkpointing*) y conmutación por fallo.
8. **Persistencia (*persistence transparency*):** Enmascara si una entidad reside en la memoria volátil o en almacenamiento persistente secundario.

Complementariamente, el sistema exhibe cuatro propiedades indispensables: **escalabilidad** para soportar crecimiento en nodos y carga sin degradación estructural; **heterogeneidad** en procesadores y redes mediante capas de *middleware*; **tolerancia a fallos** ante particiones de red; y **sincronización lógica** fundamentada en marcas de tiempo causales (relojes de Lamport) ante la ausencia de un reloj físico unificado (Silberschatz et al., 2006, §18.1, p. 605).

1.4. Análisis comparativo: ventajas, desventajas y compromisos de diseño

La adopción de esquemas distribuidos frente a entornos centralizados responde a ventajas operativas concretas (Silberschatz et al., 2006, §16.1, pp. 557–559): compartición global de periféricos y datos masivos; aceleración del cómputo mediante paralelismo y balanceo dinámico de subprocesos; resiliencia operativa ante caídas individuales de nodos; y soporte nativo para llamadas a procedimientos remotos (*remote procedure calls*, RPC).

Sin embargo, estos beneficios introducen severos compromisos de diseño (*trade-offs*) (Deitel et al., 2004, §17.2, p. 788): la sobrecarga y latencia inherente al paso de mensajes por red en lugar de buses locales; la extrema complejidad en la detección de interbloqueos distribuidos (*deadlocks*); y una superficie de ataque significativamente ampliada, donde la seguridad perimetral, la autenticación mutua y el cifrado de canales devienen imperativos.

En la Tabla 1 se sintetizan las diferencias arquitectónicas entre ambos modelos.

Tabla 1: Comparación técnica entre sistemas operativos de red (NOS) y sistemas operativos distribuidos (DO/S).

Criterio técnico	Sistema operativo de red (NOS)	Sistema operativo distribuido (DO/S)
Dominio de recursos	Recursos pertenecientes a nodos locales bajo soberanía de su núcleo anfitrión (McHoes & Flynn, 2011, §10, p. 320).	Recursos pertenecientes a un dominio global unificado gobernado por el DO/S.
Autonomía de nodos	Máxima; los hosts conservan independencia operativa y ejecución desacoplada.	Reducida; nodos estrechamente interdependientes subordinados al planificador global.
Transparencia	Baja; el usuario conoce la ubicación de servidores y protocolos de conexión (Deitel et al., 2004, §17.2, p. 787).	Total; provee una imagen de sistema único (SSI) que oculta la red física.
Memoria y procesos	Gestión puramente local; no existe migración de procesos ni memoria compartida nativa.	Gestión de memoria compartida distribuida (DSM) y migración automática de procesos.
Comunicaciones	Comunicación explícita mediante *sockets*, clientes dedicados y protocolos de red (NFS, SSH).	Comunicación transparente vía paso de mensajes del núcleo y abstracciones RPC.
Tolerancia a fallos	Resuelta a nivel de aplicación mediante configuraciones de alta disponibilidad externas.	Integrada nativamente en el núcleo con detección de fallos y reconfiguración dinámica.
Complejidad	Moderada; construido como módulos de red sobre sistemas operativos comerciales.	Muy elevada; exige núcleos especializados y sincronización formal estricta.

Históricamente, la elevada complejidad de los DO/S puros relegó sus implementaciones completas (Amoeba, Plan 9, Sprite) a la esfera investigativa. La industria consolidó un modelo híbrido pragmático: sistemas operativos de red estandarizados como base de ejecución de nodo (Linux y Windows Server), sobre los cuales plataformas de orquestación y computación en la nube (Kubernetes, OpenStack) proveen las capas de transparencia, migración y balanceo requeridas sin renunciar a la robustez de los núcleos comerciales.

2. Taxonomía de servicios y arquitecturas de servidores

El despliegue de soluciones informáticas distribuidas descansa sobre el desacoplamiento de funciones computacionales entre entidades colaboradoras. En este entorno, la provisión de capacidades de procesamiento, almacenamiento y comunicaciones se articula a través de arquitecturas estructuradas donde procesos especializados atienden requerimientos concurrentes provenientes de múltiples orígenes a través de la infraestructura de red.

2.1. Modelos cliente-servidor y paradigmas de comunicación distribuida

El modelo cliente-servidor constituye la arquitectura dominante en los sistemas distribuidos contemporáneos. Se fundamenta en una asimetría funcional estricta: el proceso **cliente** inicia la interacción solicitando la ejecución de una tarea o el acceso a un dato, mientras que el proceso **servidor** permanece a la escucha pasiva en un puerto de red predefinido, atiende la petición, procesa la transacción y devuelve un mensaje de respuesta al emisor (Silberschatz et al., 2006, §1.12, p. 26).

Desde una perspectiva analítica de ingeniería de software y sistemas operativos, cualquier aplicación distribuida descompone su funcionalidad en tres capas estructurales (Stallings, 2018, §18.1, pp. 18-5–18-6):

1. **Capa de presentación (*presentation logic*):** Responsable de la interacción con el usuario humano o con sistemas externos, formateando las pantallas, capturando eventos de entrada y desplegando los datos de salida.
2. **Capa de lógica de aplicación (*application logic*):** Contiene las reglas operativas, algoritmos de cálculo, transformaciones de negocio y validaciones que definen el comportamiento medular del sistema.
3. **Capa de gestión de datos (*data management*):** Encargada del almacenamiento físico persistente, indexación, ejecución de consultas estructuradas, control de transacciones concurrentes y preservación de la integridad referencial.

La delimitación de la frontera física donde se ejecutan estas capas sobre la infraestructura de red determina el espectro de particionamiento arquitectónico (Stallings, 2018, §18.1, pp. 18-6–18-9):

- **Procesamiento basado en host (*host-based processing*):** Modelo centralizado tradicional donde las tres capas residen íntegramente en el servidor central; la estación del usuario actúa como terminal pasiva limitada a pintar caracteres en pantalla.
- **Procesamiento basado en servidor (cliente ligero o *thin client*):** La estación cliente ejecuta exclusivamente la presentación visual básica, delegando toda la lógica de aplicación y el acceso a datos en servidores centrales con alta potencia de cómputo.
- **Procesamiento basado en cliente (cliente pesado o *fat client*):** La presentación y la lógica de aplicación completa se compilan y ejecutan en la máquina de trabajo del usuario; el servidor se restringe a actuar como motor de base de datos o almacén de ficheros compartidos.
- **Arquitecturas multinivel (tres o más capas):** Desacoplan físicamente cada capa en plataformas independientes (interfaz en navegador web o cliente móvil, lógica de negocio en servidores de aplicaciones y datos en clústeres de almacenamiento), optimizando la escalabilidad y el mantenimiento modular.

Para articular la interacción entre estos niveles, los sistemas operativos y las capas de software intermedio implementan diversos paradigmas de comunicación distribuida (Deitel et al., 2004, §17.3, pp. 788–793):

- **Paso de mensajes mediante puntos finales (sockets):** Primitiva de bajo nivel basada en descriptores de red que permite a dos procesos intercambiar flujos de bytes (orientados a conexión con TCP) o datagramas discretos (sin conexión con UDP). Requiere que el programador diseñe explícitamente el protocolo de formateo y serialización de mensajes.
- **Llamadas a procedimientos remotos (RPC):** Abstracción de alto nivel que permite invocar funciones situadas en una máquina remota empleando la misma sintaxis se-

mántica de una llamada local. El mecanismo depende de un talón de cliente (*client stub*) que empaqueta los argumentos (*marshaling*) y los transmite por la red, y un talón de servidor (*server stub*) que los desempaqueta (*unmarshaling*), ejecuta la rutina y devuelve el resultado, garantizando transparencia sintáctica (Stallings, 2018, §18.3, pp. 18-16–18-18).

- **Invocación de métodos remotos (RMI) y arquitecturas orientadas a servicios:** Extensión del paradigma RPC al modelo orientado a objetos (como Java RMI), que posibilita invocar métodos pasando referencias a objetos distribuidos, evolucionando hacia servicios web desacoplados basados en protocolos de transporte estándar (REST sobre HTTP/HTTPS, gRPC con búferes de protocolo).
- **Modelo entre iguales (*peer-to-peer*, P2P):** Paradigma simétrico donde desaparece la división estática entre cliente y servidor. Cada nodo (*peer*) posee idénticas capacidades funcionales, actuando simultáneamente como cliente que consume recursos y servidor que proporciona almacenamiento o capacidad de cálculo a la red, eliminando cuellos de botella centralizados (Silberschatz et al., 2006, §1.12, pp. 27–28).

2.2. Clasificación funcional de servidores en redes corporativas

En los entornos de producción modernos, la especialización funcional de las máquinas servidoras permite garantizar el aislamiento de procesos, maximizar la eficiencia en el aprovechamiento de recursos físicos (CPU, memoria RAM, buses de almacenamiento y ancho de banda de red) y consolidar directivas de seguridad uniformes. Conforme a las demandas de infraestructura y a los servicios que proveen a los sistemas operativos clientes, los servidores se clasifican técnicamente en las siguientes categorías cardinales:

1. **Infraestructura básica y direccionamiento:** Proveen resolución de nombres y parámetros de red en capas 3 y 4. Incluyen servidores DNS (traducción jerárquica de nombres a direcciones IP) y servidores DHCP (concesión y renovación automática de direcciones IP, máscaras y pasarelas).
2. **Almacenamiento y archivos:** Abstraen unidades secundarias mediante sistemas de archivos de red (NFS, SMB/CIFS) o almacenamiento en bloque (iSCSI, Fibre Channel). Implementan control de concurrencia, cuotas, listas ACL, instantáneas (*snapshots*) y arreglos RAID (McHoes & Flynn, 2011, §10, pp. 325–326).
3. **Gestión de impresión:** Coordinan el acceso concurrente a periféricos de salida mediante colas en disco (*spooling*) y traducción de flujos (PDF, PostScript, PCL) usando protocolos estándar como IPP o LPD/LPR.
4. **Directorio, identidad y autenticación:** Centralizan credenciales, perfiles y directivas de seguridad corporativas mediante LDAP, Kerberos (emisión de boletos de autenticación mutua) o RADIUS (autorización centralizada de accesos de red).
5. **Seguridad perimetral y enrutamiento:** Regulan el flujo entre zonas de red dispares mediante cortafuegos con seguimiento de estados (*stateful inspection*), traducción NAT para aislar direccionamiento privado (RFC 1918) y servidores proxy con almacenamiento en caché e inspección de contenido.
6. **Aplicaciones y bases de datos:** Procesan la lógica de negocio y transacciones persistentes. Integran servidores web (Apache, Nginx), motores de ejecución (Node.js, PHP-FPM) y sistemas RDBMS (PostgreSQL, MariaDB, Oracle) bajo garantías ACID.
7. **Acceso remoto y comunicaciones (RAS):** Proveen conectividad segura a clientes remotos hacia la LAN interna mediante túneles cifrados de red privada virtual (VPN) basados en protocolos como IPsec, OpenVPN o WireGuard.

Para sintetizar la articulación funcional de estos componentes dentro de la infraestructura tecnológica, en la Tabla 2 se presenta una matriz comparativa que clasifica los servidores según su rol operativo, los protocolos cardinales involucrados y su impacto en la administración de recursos del sistema operativo anfitrión.

Tabla 2: Clasificación funcional de servidores en redes corporativas.

Categoría de servidor	Protocolos y estándares cardinales	Recursos críticos del sistema operativo
Infraestructura de red	DNS (puerto 53 UDP/TCP), DHCP (puertos 67/68 UDP), NTP, IPAM.	Baja latencia en resolución de sockets, estabilidad en interfaces de red e integridad en tablas de concesiones.
Almacenamiento y archivos	NFSv4 (puerto 2049 TCP), SMB/CIFS (puerto 445 TCP), iSCSI (puerto 3260 TCP).	Búferes de caché en memoria de páginas, alto rendimiento de E/S en disco (*IOPS*) y gestión de inodos/ACL.
Gestión de impresión	IPP (puerto 631 TCP), LPD/LPR (puerto 515 TCP), filtros PPD/Ghostscript.	Espacio de almacenamiento temporal para colas de ficheros en disco (*spool*) y aislamiento de filtros de rasterización.
Directorio y autenticación	LDAP/LDAPS (puertos 389/636 TCP), Kerberos (puerto 88 UDP/TCP), RADIUS (puertos 1812/1813 UDP).	Sincronización estricta de relojes lógicos, cifrado en memoria y rapidez de consulta en estructuras arbóreas indexadas.
Seguridad y perímetro	Filtrado *netfilter*/nftables, NAT (RFC 3022), HTTP/SOCKS en proxies (Squid).	Eficiencia del núcleo en inspección de paquetes en anillo 0, tablas de seguimiento de conexiones y memoria de caché.
Aplicaciones y datos	HTTP/HTTPS (puertos 80/443 TCP), FastCGI, SQL (PostgreSQL, MariaDB).	Asignación masiva de memoria RAM compartida para cachés de base de datos, múltiples hilos de CPU y descriptores abiertos.
Acceso remoto (RAS)	IPsec (IKEv2, puertos 500/4500 UDP), OpenVPN (puerto 1194 UDP), WireGuard (UDP).	Interrupciones por aceleración criptográfica (AES-NI), interfaces de red virtuales (TUN/TAP) y control de enrutamiento.

Esta segmentación funcional evidencia que un servidor contemporáneo no se define primordialmente por su arquitectura física de hardware, sino por la naturaleza de los demonios y servicios que ejecuta en su espacio de usuario y en su núcleo, así como por las políticas de asignación de recursos que el sistema operativo implementa para garantizar su disponibilidad y resiliencia en la red.

3. Sistemas operativos orientados a servidores

El funcionamiento continuo de una infraestructura de red corporativa depende de la estabilidad, seguridad y rendimiento del sistema operativo que gobierna el hardware de cada estación servidora. A diferencia de las distribuciones diseñadas para estaciones de trabajo cliente o consumo personal, un sistema operativo para servidor está especializado en gestionar concurrencia masiva, maximizar el aprovechamiento de los recursos físicos de cálculo y mitigar cualquier evento que comprometa la disponibilidad del servicio.

3.1. Arquitectura y especialización de sistemas operativos para servidor

La distinción entre un sistema operativo de escritorio (*desktop*) y uno orientado a servidores no reside necesariamente en las llamadas al sistema fundamentales, sino en las políticas de configuración del núcleo, la asignación de memoria, la planificación de la CPU y la omisión deliberada de subsistemas innecesarios:

1. **Políticas del planificador de la CPU:** En un sistema de escritorio, el planificador prioriza la latencia de respuesta interactiva para que las aplicaciones de la interfaz gráfica respondan inmediatamente a eventos de teclado y ratón. En contraste, un núcleo de servidor optimiza el rendimiento global (*throughput*) y la equidad en el tiempo de procesamiento, minimizando los cambios de contexto costosos entre los miles de hilos de trabajo de bases de datos o servidores web concurrentes (Stallings, 2018, §18.1, pp. 18-3–18-4).
2. **Gestión de memoria física y caché de páginas:** Un servidor dedica la mayor parte de su memoria RAM a la caché de páginas del núcleo (*page cache*) para acelerar las lecturas y escrituras en disco. Además, ajusta la agresividad del intercambio de memoria (*swappiness*) para evitar la expulsión prematura de páginas activas de procesos transaccionales hacia el almacenamiento secundario.
3. **Subsistema de red y colas de interrupción:** Los servidores implementan técnicas de alto rendimiento como el escalado lateral de recepción (RSS, *Receive Side Scaling*) para distribuir el procesamiento de paquetes entrantes entre múltiples núcleos físicos de CPU, combinadas con llamadas asíncronas no bloqueantes de multiplexación de descriptores (como `epoll` en Linux, `kqueue` en BSD y puertos de finalización de E/S o *IOCP* en Windows NT), permitiendo atender decenas de miles de conexiones simultáneas por proceso.
4. **Confiabilidad del almacenamiento secundario:** Incorporan soporte nativo para sistemas de archivos transaccionales avanzados que implementan comprobación de redundancia de datos (*checksumming*), instantáneas de volumen (snapshots), compresión en línea y administración de volúmenes lógicos con tolerancia a fallos mediante arreglos redundantes de discos independientes (RAID) por hardware o software.
5. **Minimización de superficie de ataque:** Los sistemas de servidor prescinden por defecto de servidores gráficos (X11 o Wayland) y entornos de escritorio, operando exclusivamente en modo terminal de línea de comandos (CLI) administrada por SSH o interfaces remotas declarativas, lo que reduce drásticamente el consumo de memoria en reposo y cierra vectores de vulnerabilidad.

3.2. Estudio técnico y comparativo de diez plataformas de servidor

A continuación se analizan diez plataformas representativas empleadas en la industria, detallando su arquitectura de núcleo, mecanismos de seguridad, subsistemas de almacenamiento y áreas cardinales de aplicación.

3.2.1. Microsoft Windows Server

Basado en la arquitectura del núcleo Windows NT (núcleo híbrido), desacopla el hardware mediante la capa HAL y ejecuta sus subsistemas ejecutivos en anillo 0 (Microsoft Corporation, 2024). Es el pilar de redes corporativas mediante Active Directory (AD DS), integrando autenticación Kerberos, directivas de grupo (GPO), virtualización Hyper-V y sistemas de archivos NTFS y ReFS (*Resilient File System*), administrado mediante PowerShell y Windows Admin Center.

3.2.2. Red Hat Enterprise Linux (RHEL)

Estándar comercial de referencia para entornos de misión crítica con ciclo de vida extendido de diez años. Basado en un núcleo monolítico modular de Linux, destaca por la integración nativa de SELinux (*Security-Enhanced Linux*) en modo forzoso por defecto (control de acceso obligatorio, MAC) (Red Hat, Inc., 2024), gestor DNF/RPM, sistema de archivos transaccional XFS y despliegue de contenedores OCI mediante Podman y Buildah.

3.2.3. Debian GNU/Linux

Sistema operativo universal sustentado en gobernanza comunitaria abierta y rigor técnico extremo en su rama estable (The Debian Project, 2023). Utiliza el sistema de paquetes APT (`dpkg`), ofrece un consumo de memoria basal mínimo en modo servidor y garantiza máxima previsibilidad operativa, consolidándose como la base troncal de múltiples distribuciones industriales.

3.2.4. Ubuntu Server

Variante empresarial mantenida por Canonical derivada de Debian con ciclo predecible de versiones LTS cada dos años y hasta diez años de soporte extendido (Canonical Ltd., 2024). Se encuentra optimizada para cómputo en la nube e infraestructuras de microservicios, integrando configuración de red con Netplan, ZFS en Linux, paquetes aislados Snap y aprovisionamiento desatendido con MAAS.

3.2.5. SUSE Linux Enterprise Server (SLES)

Distribución corporativa diseñada para cómputo intensivo y centros de datos de misión crítica, certificada para bases de datos en memoria SAP HANA (SUSE LLC, 2023). Integra la herramienta centralizada de administración YaST, paquetería RPM con Zypper y el sistema de archivos Btrfs con instantáneas automáticas Snapper para recuperación inmediata ante actualizaciones.

3.2.6. FreeBSD

Sistema operativo UNIX descendiente directo del código de investigación BSD de UC Berkeley, que desarrolla su núcleo modular y espacio de usuario como un sistema unificado (The FreeBSD Project, 2023). Sobresale por su pila de red de muy alto rendimiento, almacenamiento de 128 bits OpenZFS nativo con autorreparación y virtualización ligera mediante jaulas (*FreeBSD Jails*) para aislamiento sin hipervisor.

3.2.7. Oracle Solaris / illumos

Sistema operativo UNIX SVR4 certificado, diseñado para servidores multiprocesador masivos (Oracle Corporation, 2023). Plataforma pionera que introdujo el sistema transaccional ZFS con sumas de verificación, el marco de observabilidad DTrace, zonas de aislamiento (*Solaris Zones*) y el gestor tolerante a fallos SMF, tecnologías cuyo desarrollo abierto continúa en el proyecto illumos.

3.2.8. OpenBSD

Sistema de la familia BSD enfocado prioritariamente en máxima seguridad proactiva y auditoría técnica permanente del código fuente (The OpenBSD Project, 2024). Creador de OpenSSH y del cortafuegos Packet Filter (PF), implementa mitigaciones de memoria avanzadas en su núcleo (WˆX, ASLR aleatorio) y llamadas restrictivas `pledge` y `unveil` para confinamiento estricto de procesos.

3.2.9. Rocky Linux / AlmaLinux

Distribuciones comunitarias surgidas para preservar la compatibilidad binaria 1:1 con RHEL tras el cambio de enfoque de CentOS hacia CentOS Stream (Rocky Enterprise Software Foundation, 2024). Proporcionan la robustez, el gestor DNF/RPM, el confinamiento de SELinux y el soporte de aplicaciones corporativas de grado RHEL de manera libre y sin costos de licenciamiento.

3.2.10. VMware ESXi

Hipervisor de tipo 1 (*bare-metal*) especializado en la virtualización de servidores en centros de datos, ejecutándose directamente sobre el silicio del equipo sin un sistema operativo anfitrión intermedio (VMware, Inc., 2023). Su núcleo propietario VMkernel administra directamente vCPUs, compresión y deduplicación de memoria RAM, y almacenamiento VMFS, maximizando la densidad de consolidación virtual.

3.2.11. Matriz comparativa de sistemas operativos de servidor

Para estructurar las diferencias arquitectónicas y operativas de estas diez plataformas, en la Tabla 3 se sintetiza su diseño interno, mecanismos de aislamiento y entornos de aplicación principales.

Tabla 3: Comparación técnica de diez plataformas de sistema operativo orientadas a servidores.

Sistema operativo	Núcleo y familia	Almacenamiento nativo	Aislamiento y seguridad	Nicho corporativo principal
Windows Server	Núcleo híbrido (familia Windows NT).	NTFS, ReFS, espacios de almacenamiento directo.	Listas ACL de Windows, aislamiento Hyper-V, directivas GPO.	Servicios de directorio Active Directory, aplicaciones .NET, servidores corporativos Windows.
RHEL	Monolítico modular (GNU/Linux).	XFS (predeterminado), ext4, LVM.	SELinux forzoso (MAC), cortafuegos *firewalld*, contenedores Podman.	Cargas de misión crítica corporativa, centros de datos empresariales y nubes privadas.
Debian	Monolítico modular (GNU/Linux).	ext4, XFS, Btrfs, soporte ZFS.	Permisos POSIX, AppArmor/SELinux configurable, paquetería APT estricta.	Infraestructura de servidores web de alta estabilidad, servicios comunitarios y base de sistemas.
Ubuntu Server	Monolítico modular (GNU/Linux).	ext4, ZFS en Linux, LVM.	Perfiles AppArmor, confinamiento Snap, actualizaciones automáticas de seguridad.	Nube pública (AWS, Azure, GCP), entornos de contenedores Kubernetes y microservicios.
SLES	Monolítico modular (GNU/Linux).	Btrfs con instantáneas Snapper, XFS.	AppArmor, configuración integral YaST, alta disponibilidad con clúster Pacemaker.	Plataformas ERP empresariales, bases de datos SAP HANA y grandes clústeres de computación.
FreeBSD	Monolítico modular (UNIX/BSD).	OpenZFS (128 bits con autorreparación), UFS2.	Jaulas (*Jails*) para virtualización ligera de SO, cortafuegos Packet Filter (PF).	Infraestructuras de red de alta tasa de transferencia, enrutadores perimetrales y servidores ZFS.
Solaris / **illumos**	Monolítico modular simétrico (UNIX SVR4).	ZFS nativo con compresión e instantáneas.	Zonas de aislamiento (Solaris Zones), observabilidad DTrace, RBAC.	Servidores multiprocesador masivos, bases de datos Oracle de alto rendimiento y almacenamiento ZFS.
OpenBSD	Monolítico (UNIX/BSD).	FFS2 (*Fast File System*), softupdates.	Mitigaciones W^X, ASLR aleatorio, llamadas `pledge`/`unveil`, cortafuegos PF.	Cortafuegos perimetrales, concentradores de VPN, servidores DNS/BGP y entornos de alta seguridad.
Rocky / **AlmaLinux**	Monolítico modular (GNU/Linux).	XFS, ext4, LVM estructurado.	SELinux en modo forzoso, compatibilidad binaria 1:1 con directivas RHEL.	Reemplazo corporativo de CentOS, entornos universitarios y centros de cómputo sin costo de suscripción.
VMware ESXi	Micronúcleo propietario especializado (VMkernel).	VMFS (*Virtual Machine File System*), vSAN.	Aislamiento de hardware en anillo 0, segmentación de vSwitch y cifrado de VM.	Consolidación masiva de servidores virtuales en centros de datos corporativos (hipervisor tipo 1).

4. Servidores de conectividad, enrutamiento y seguridad perimetral

La operación continua de una red corporativa demanda mecanismos de control de tráfico en las capas de red, transporte y aplicación. Los servidores de conectividad y seguridad perimetral regulan los flujos de paquetes que transitan entre segmentos internos y redes públicas desconfiadas, mitigando vectores de ataque, optimizando el espacio de direccio-

namiento y dinamizando la asignación de parámetros de red.

4.1. Servidores de traducción de direcciones de red (NAT)

La traducción de direcciones de red (*Network Address Translation*, NAT), formalizada en el RFC 3022, permite a un enrutador o pasarela perimetral modificar en tránsito las cabeceras de los paquetes IP, mapeando direcciones privadas no enrutables (RFC 1918) a direcciones IP públicas globalmente enrutables (Srisuresh & Egevang, 2001, §1, pp. 1–3). Esta tecnología surgió para mitigar el agotamiento de direcciones IPv4 de 32 bits, consolidando a la vez un aislamiento estructural que oculta la topología interna.

En el núcleo del sistema operativo anfitrión, el módulo NAT opera en anillo 0 interceptando los paquetes en los ganchos de preenrutamiento (*prerouting*) o postenrutamiento (*postrouting*). Se implementa bajo tres esquemas operativos:

- **NAT estático:** Correspondencia biunívoca permanente (1:1) entre una IP privada y una pública para exponer servidores internos sin revelar su subred real.
- **NAT dinámico:** Asignación temporal de una dirección pública extraída de un grupo (*pool*) reservado mientras dure la sesión del host.
- **Traducción de direcciones y puertos (NAPT/PAT):** Esquema masivo donde múltiples hosts internos comparten una sola IP pública mediante sobrecarga de puertos en la capa de transporte (Srisuresh & Egevang, 2001, §4, pp. 6–8), reescribiendo la IP origen y sustituyendo el puerto efímero local por uno asignado dinámicamente en el servidor.

El seguimiento de conexiones se sustenta en una tabla en la memoria del núcleo (módulo `nf_conntrack` en Linux), que indexa la tupla de cinco elementos: IP origen, puerto origen, IP destino, puerto destino y protocolo. Al retornar el tráfico, el núcleo sustituye la cabecera, recalcula las sumas de comprobación (*checksums*) de IP y TCP/UDP en tiempo real y conmuta el paquete hacia la subred local. Para protocolos de capa 7 que incrustan direcciones IP en su carga útil (como FTP activo o SIP), el núcleo invoca pasarelas a nivel de aplicación (ALG, *Application Layer Gateway*) que reescriben los campos internos de señalización.

4.2. Servidores de filtrado y cortafuegos (Firewall)

Un cortafuegos (*firewall*) aplica directivas de control de acceso entre zonas de confianza dispares, inspeccionando el tráfico bidireccional para autorizar, rechazar o descartar tramas conforme a políticas formales (Deitel et al., 2004, §19.3, pp. 942–944). Arquitectónicamente, se diferencian tres niveles de inspección:

1. **Filtrado sin estado (*stateless*):** Examina cabeceras aisladas en capas 3 y 4 (IP origen/destino, puerto y banderas TCP como SYN, ACK o RST) sin memoria de flujos previos, siendo vulnerable a suplantaciones de identidad.
2. **Inspección con estado (*stateful packet inspection*, SPI):** Modela los flujos mediante máquinas de estado finito en el núcleo. Al validar el saludo de tres vías de TCP, cataloga la conexión como `NEW` y luego `ESTABLISHED` (Deitel et al., 2004, §19.3, p. 943); los paquetes de respuesta válidos se conmutan de forma inmediata y el tráfico no solicitado se descarta.
3. **Cortafuegos de aplicación (WAF / NGFW):** Reensambla flujos en capa 7 para analizar la semántica de protocolos web (HTTP/HTTPS, DNS), ejecutando inspección profunda de paquetes (*Deep Packet Inspection*, DPI) contra inyecciones y anomalías de carga útil.

En el espacio de núcleo de Linux, el marco de trabajo netfilter provee los puntos de enganche `PREROUTING`, `INPUT`, `FORWARD`, `OUTPUT` y `POSTROUTING`, operados por el subsistema moderno `nftables`, que ejecuta las reglas como código de máquina virtual de bytes en anillo 0 para minimizar la sobrecarga de interrupciones de CPU. En entornos BSD, Packet Filter (PF) ejecuta normalización de tráfico (*scrubbing*), resolviendo la desfragmentación de paquetes y el saneamiento de banderas TCP antes de la evaluación de políticas.

4.3. Servidores de configuración dinámica de direcciones (DHCP)

El protocolo de configuración dinámica de host (*Dynamic Host Configuration Protocol*, DHCP), especificado en el RFC 2131, automatiza la asignación centralizada de direccionamiento y parámetros de red a clientes locales, evitando colisiones por duplicidad de IP (Droms, 1997, §1, pp. 1–4).

La negociación de parámetros opera en espacio de usuario sobre sockets UDP (puertos 67 en el servidor y 68 en el cliente) mediante el ciclo DORA de cuatro fases:

1. **DHCPDISCOVER:** Difusión masiva (*broadcast* en capa 2 `ff:ff:ff:ff:ff:ff` y capa 3 `255.255.255.255`) emitida por el cliente con su dirección MAC física para localizar servidores disponibles.
2. **DHCPOFFER:** Los servidores receptores seleccionan una IP libre de su ámbito (*scope*) y responden con una oferta que detalla la IP propuesta, la máscara de subred y el tiempo de concesión (*lease time*).
3. **DHCPREQUEST:** El cliente selecciona formalmente una oferta y difunde su aceptación para notificar a los servidores y liberar las reservas descartadas.
4. **DHCPACK:** El servidor seleccionado confirma la concesión e inserta las opciones RFC 2132: pasarela predeterminada (*default gateway*), servidores DNS, sufijo de búsqueda y servidores NTP (Droms, 1997, §3.1, pp. 13–16).

El cliente gestiona la vida del alquiler con dos temporizadores: T1 (al 50 % del tiempo, renegociación unicast directa) y T2 (al 87.5 % del tiempo, renegociación en difusión si el servidor original no responde). Para atravesar enrutadores en redes segmentadas por VLAN, se despliegan agentes de retransmisión DHCP (*DHCP Relay Agents*, RFC 3046). La seguridad del servicio se refuerza mediante inspección DHCP (*DHCP Snooping*) en conmutadores, bloqueando respuestas originadas en puertos no confiables para neutralizar servidores falsos (*rogue DHCP*).

4.4. Servidores de intermediación y almacenamiento en caché (Proxy)

Un servidor proxy es un intermediario en la capa de aplicación que desacopla la comunicación directa entre clientes y servidores remotos, evaluando políticas de acceso y gestionando solicitudes independientes a su propio nombre. Estructuralmente se clasifica en tres arquitecturas:

- **Proxy de reenvío (*forward proxy*):** Ubicado en la frontera de salida corporativa. Centraliza la navegación, aplica filtrado de contenidos URL, anonimiza las IP internas y almacena en caché intermedia objetos web estáticos, reduciendo el consumo de ancho de banda.
- **Proxy transparente (*transparent proxy*):** Intercepta el tráfico web saliente en la pasarela perimetral sin configuración en los clientes mediante redirección de puertos TCP 80/443 con reglas NAT en el cortafuegos.

- **Proxy inverso (*reverse proxy*):** Desplegado por delante de granjas de servidores web internos (mediante motores como Nginx, HAProxy o Envoy). Desempeña cuatro funciones críticas: balanceo de carga backend (Round Robin ponderado, menor número de conexiones, dispersión de IP); terminación de cifrado SSL/TLS para descargar de cómputo asimétrico al backend; aceleración de caché estática en memoria RAM con compresión dinámica (Gzip, Brotli); y mitigación de ataques DDoS mediante limitación de tasa de peticiones (*rate limiting*).

En la Tabla 4 se sintetizan los atributos de estas cuatro plataformas de conectividad y seguridad perimetral.

Tabla 4: Comparación técnica de servidores de conectividad, enrutamiento y seguridad perimetral.

Servidor	Capa de operación	Mecanismo en el núcleo del SO	Aporte cardinal a la infraestructura
NAT	Capas 3 y 4 (Red y Transporte).	Reescritura de cabeceras IP/puertos y tablas de seguimiento de conexiones (`conntrack`) en anillo 0 (Srisuresh & Egevang, 2001, §1, p. 2).	Mitigación del agotamiento IPv4, reutilización de rangos RFC 1918 y ocultamiento de topología interna.
Firewall	Capas 3, 4 y 7 (Filtrado con estado y DPI).	Ganchos de intercepción (netfilter/PF), tablas de estados y máquinas de estados finitos (Deitel et al., 2004, §19.3, p. 943).	Aislamiento de zonas de seguridad, control granular de políticas de tráfico y prevención de intrusiones.
DHCP	Capa de aplicación (sobre UDP 67/68 en Capa 4).	Manejo de sockets de difusión (broadcast) en espacio de usuario y enlace con tablas de arrendamiento en disco (Droms, 1997, §1, p. 3).	Automatización en la asignación de parámetros IP, eliminación de colisiones y administración de leases.
Proxy	Capa 7 (Aplicación, HTTP/HTTPS, SOCKS).	Desacoplamiento de sockets en espacio de usuario, memoria intermedia de caché y multiplexación asíncrona no bloqueante.	Almacenamiento en caché, filtrado de URL, balanceo de carga backend, terminación SSL/TLS y mitigación DDoS.

5. Servidores de autenticación, compartición, impresión y aplicaciones

La operatividad de los sistemas distribuidos demanda servicios especializados que garanticen la identidad de los usuarios, faciliten la interoperabilidad de archivos entre plataformas dispares, gestionen la salida documental y sustenten la ejecución de aplicaciones

empresariales. A continuación se analizan cinco arquitecturas de servidor cardinales en redes corporativas.

5.1. Servidores de autenticación y contabilidad de accesos (RADIUS)

El servicio de usuario de acceso telefónico de autenticación remota (*Remote Authentication Dial-In User Service*, RADIUS), estandarizado en el RFC 2865 y extendido para contabilidad en el RFC 2866, es un protocolo de red cliente-servidor que centraliza la administración de seguridad bajo el modelo AAA: Autenticación (*Authentication*, verificación de identidad), Autorización (*Authorization*, concesión de privilegios y políticas) y Contabilidad (*Accounting*, registro de tiempos de conexión y consumo de datos) (Rigney et al., 2000, §1, pp. 1–4).

RADIUS opera sobre el protocolo de transporte UDP empleando los puertos oficiales 1812 para autenticación/autorización y 1813 para contabilidad. En su topología, los equipos clientes de RADIUS son servidores de acceso a la red (NAS, *Network Access Server*), conmutadores compatibles con el estándar IEEE 802.1X, puntos de acceso Wi-Fi empresariales (WPA2/WPA3 Enterprise) o concentradores VPN. Cuando un usuario solicita acceso, el NAS empaqueta las credenciales en una trama `Access-Request` y la transmite al servidor RADIUS (como FreeRADIUS o Microsoft Network Policy Server, NPS).

El servidor RADIUS valida las credenciales contra almacenes de identidad corporativos (Active Directory, LDAP o bases de datos SQL relacionales) y responde con un mensaje `Access-Accept` (acompañado de atributos de autorización como VLAN asignada o filtros de tráfico), `Access-Reject` (denegación de acceso) o `Access-Challenge` (solicitud de un segundo factor de autenticación) (Rigney et al., 2000, §2, pp. 6–10). Los mensajes RADIUS emplean pares atributo-valor (AVP, *Attribute-Value Pairs*) y protegen la integridad de las contraseñas mediante funciones criptográficas de dispersión y un secreto compartido (*shared secret*) entre el NAS y el servidor, evitando exponer las bases de datos de usuarios en los dispositivos de borde.

5.2. Servidores de interoperabilidad y compartición de archivos (SAMBA)

Samba es una suite de software de código abierto que implementa de forma nativa los protocolos de red SMB/CIFS (*Server Message Block / Common Internet File System*) en sistemas operativos tipo UNIX, GNU/Linux y BSD, posibilitando una interoperabilidad fluida con clientes de la familia Microsoft Windows (The Samba Team, 2024).

La arquitectura de Samba provee dos capacidades cardinales en la infraestructura corporativa: como servidor de archivos e impresión, permite a clientes Windows montar carpetas remotas alojadas en volúmenes Linux gestionando el bloqueo concurrente (*record locking*) y mapeando listas de control de acceso NTFS (ACL) sobre permisos POSIX; y como controlador de dominio Active Directory (AD DC a partir de Samba 4), emula los servicios de Microsoft mediante un servidor LDAP integrado, un centro Kerberos (KDC) y replicación de directivas de grupo (GPO). En el sistema operativo anfitrión, su arquitectura descansa en tres demonios en espacio de usuario: `smbd` gestiona las conexiones sobre el puerto TCP 445 y atiende las operaciones de lectura/escritura de ficheros; `nmbd` resuelve nombres NetBIOS sobre TCP/IP y anuncia servicios mediante el protocolo WINS (puertos UDP 137 y 138); y `winbindd` vincula los módulos PAM y NSS locales con Active Directory,

unificando identidades corporativas con cuentas de usuario UNIX nativas.

5.3. Servidores de gestión y colas de impresión (CUPS)

El sistema común de impresión para UNIX (*Common UNIX Printing System*, CUPS), mantenido bajo el proyecto OpenPrinting, es el estándar modular de gestión de impresión en sistemas operativos GNU/Linux, macOS y BSD (The OpenPrinting Project, 2024). CUPS reemplazó a los antiguos sistemas LPD/LPR mediante la adopción integral del protocolo de impresión de Internet (IPP, *Internet Printing Protocol*, estandarizado en los RFC 8010 y RFC 8011), operando sobre el protocolo HTTP en el puerto TCP 631.

La arquitectura de CUPS se compone de tres módulos funcionales en cascada: el planificador central (`cupsd`), demonio que atiende peticiones IPP, gestiona colas en disco (spooling) y supervisa periféricos; el sistema de filtros modulares, que traduce documentos en formatos independientes (PDF, PostScript o texto) al lenguaje nativo de la impresora (PCL, ESC/P o mapas de bits rasterizados mediante Ghostscript y especificaciones PPD); y los procesos de transporte (*backends*), ejecutables que transmiten los flujos procesados hacia puertos físicos (USB, serie) o a través de la red hacia impresoras remotas empleando protocolos como AppSocket/JetDirect (puerto TCP 9100), IPP, LPD (puerto TCP 515) o recursos compartidos SMB vía Samba.

5.4. Servidores de acceso remoto y pasarelas de conexión (RAS)

Un servidor de acceso remoto (*Remote Access Server*, RAS) es una plataforma de red que actúa como pasarela segura para conectar estaciones de trabajo cliente remotas a la infraestructura corporativa interna, otorgándoles acceso a los recursos locales como si estuvieran físicamente conectadas a la red LAN (Patel et al., 2001, §1, pp. 1–3). Evolutivamente, los servidores RAS transitaron desde plataformas de marcación analógica por módem (*dial-in*) sobre líneas telefónicas conmutadas (PSTN/ISDN) empleando el protocolo PPP, hacia servidores de redes privadas virtuales (VPN) que establecen túneles seguros y cifrados sobre redes públicas de paquetes.

En la actualidad, un servidor RAS implementa arquitecturas de túnel basadas en tres tecnologías criptográficas principales: L2TP sobre IPsec (RFC 3193), donde L2TP encapsula tramas PPP y la suite IPsec provee autenticación mutua, negociación de claves (IKEv2) y cifrado de carga útil con el protocolo ESP (Patel et al., 2001, §4, pp. 7–10); redes privadas basadas en SSL/TLS (OpenVPN), operando en espacio de usuario sobre interfaces virtuales del núcleo como adaptadores TUN (enrutamiento IP en capa 3) o TAP (puenteo Ethernet en capa 2); y WireGuard, protocolo integrado directamente en el espacio de núcleo de Linux y BSD que utiliza criptografía contemporánea de curva elíptica (Curve25519, ChaCha20-Poly1305), brindando mínimas latencias y conmutación transparente entre interfaces de red móviles. El servidor RAS administra las tablas de enrutamiento locales y federa la autenticación contra servidores RADIUS o directorios corporativos LDAP/Active Directory.

5.5. Servidores de infraestructura web integrada (pila LAMP)

El concepto LAMP (acrónimo de Linux, Apache, MySQL/MariaDB y PHP/Perl/Python) define la pila de software arquetípica de cuatro capas para el desarrollo y despliegue de aplicaciones web dinámicas: el sistema operativo Linux administra recursos de cómputo,

memoria virtual y sockets TCP/IP en anillo 0; el servidor Apache HTTP procesa peticiones web sobre puertos TCP 80 y 443 mediante módulos multiproceso asíncronos (MPM `event`) para soportar conexiones persistentes con bajo consumo de memoria (The Apache Software Foundation, 2024); el gestor de bases de datos relacionales MySQL/MariaDB provee persistencia estructurada transaccional bajo ACID mediante motores de almacenamiento como InnoDB (con aislamiento multiversión MVCC y *buffer pool* en RAM); y el intérprete PHP (mediante PHP-FPM) ejecuta la lógica de aplicación como un servicio desacoplado comunicado con Apache a través de sockets UNIX locales de alta velocidad.

En la Tabla 5 se resumen las características de estas cinco plataformas de servicios de red, destacando su capa operativa, demonios cardinales y función en la infraestructura informática.

Tabla 5: Comparación técnica de servidores de autenticación, compartición, impresión y aplicaciones.

Servidor / Pila	**Protocolos y estándares**	**Demonios y procesos en el SO**	**Función cardinal en la red**
RADIUS	AAA sobre UDP (puertos 1812/1813), RFC 2865, IEEE 802.1X.	Demonio servidor (`radiusd` / FreeRADIUS), integración PAM/SQL.	Centralización de autenticación, autorización y contabilidad de accesos de red corporativos.
SAMBA	SMB/CIFS (puerto 445 TCP), NetBIOS (puertos 137/138 UDP).	`smbd` (archivos/SMB), `nmbd` (nombres) y `winbindd` (integración AD).	Interoperabilidad de sistemas de archivos, compartición de carpetas y controlador de dominio Active Directory.
CUPS	IPP (RFC 8010/8011 sobre HTTP puerto 631 TCP), PPD, PostScript.	Demonio planificador `cupsd`, filtros de rasterización y backends.	Gestión centralizada de colas de impresión en disco (spooling), conversión de formatos y despacho.
RAS	L2TP/IPsec (RFC 3193), OpenVPN (SSL/TLS), WireGuard.	Demonios VPN, interfaces de red virtuales del núcleo (`tun`/`tap`, WireGuard).	Pasarela de acceso remoto cifrado para conectar usuarios móviles a la red corporativa interna.
LAMP	HTTP/HTTPS (puertos 80/443 TCP), SQL, FastCGI.	Demonio web (`httpd`/`apache2`), motor RDBMS (`mysqld`) y PHP-FPM.	Plataforma integrada de cuatro capas para alojamiento y ejecución de aplicaciones web dinámicas.

6. Herramientas de entorno gráfico, desarrollo y capas de compatibilidad

Junto a los servicios de red y almacenamiento desatendidos, los sistemas operativos contemporáneos integran subsistemas y utilidades que optimizan la interfaz de usuario, proporcionan entornos ágiles de desarrollo de software y facilitan la interoperabilidad binaria entre plataformas dispares. A continuación se analiza la arquitectura y el propósito de tres herramientas cardinales en entornos abiertos: Compiz, Geany y Wine.

6.1. Gestores de composición de ventanas: Compiz

Compiz es un gestor de composición de ventanas (*compositing window manager*) para el sistema de ventanas X (*X Window System*, X11), concebido originalmente por David Reveman en Novell bajo los auspicios de Freedesktop.org (Freedesktop.org, 2023). Su aporte arquitectónico fundamental consistió en sustituir el modelo de renderizado bidimensional directo y destructivo de X11 por un paradigma de composición acelerado por hardware mediante la interfaz gráfica OpenGL. En los gestores tradicionales de X11, cuando una ventana se mueve o se superpone, el servidor X solicita a las aplicaciones redibujar las regiones dañadas, lo que genera artefactos visuales y sobrecarga de CPU. Compiz elude este cuello de botella mediante la extensión Composite de X11, redirigiendo el dibujo de cada ventana hacia mapas de bits fuera de pantalla (*off-screen pixmaps*). Utilizando la extensión `GLX_EXT_texture_from_pixmap` (implementada sobre arquitecturas AIGLX o Xgl), Compiz vincula dichos pixmaps directamente como texturas bidimensionales y tridimensionales en la memoria de la tarjeta de video (GPU).

A nivel estructural, Compiz se divide en un núcleo liviano (`src/`) y una arquitectura modular de complementos gestionados dinámicamente (*plugins* enlazados mediante la biblioteca `libcompizconfig` y configurables vía D-Bus). Esta modularidad permite ejecutar transformaciones geométricas matriciales en tiempo real sobre las ventanas tratadas como mallas poligonales: proyecta los múltiples escritorios virtuales sobre las caras de un prisma tridimensional navegable mediante rotación espacial interactiva (cubo de escritorio o *desktop cube*); modela las ventanas como masas interconectadas por resortes bidimensionales que reaccionan con deformación elástica al arrastre del cursor (*wobbly windows*); y genera desenfoque gaussiano de fondos (*blur*), sombras difusas, miniaturas dinámicas y transiciones de desvanecimiento mediante sombreadores (*shaders*) de la GPU. Su propósito cardinal radica en desacoplar el servidor gráfico del dibujo de ventanas, trasladar la carga de renderizado a la GPU y proveer retroalimentación espacial intuitiva que optimiza la productividad y ergonomía visual en estaciones de trabajo multitarea.

6.2. Entornos ligeros de desarrollo integrado: Geany

Geany es un entorno de desarrollo integrado (*Integrated Development Environment*, IDE) ligero y multiplataforma, concebido para brindar capacidades completas de programación sin incurrir en la sobrecarga de dependencias ni el consumo excesivo de memoria propios de los entornos de escritorio integrados (The Geany Project, 2024). Su arquitectura se fundamenta en el desacoplamiento entre la interfaz gráfica y el motor de manipulación textual: utiliza el conjunto de herramientas GIMP Toolkit (GTK+) para la capa visual, lo que le permite prescindir de bibliotecas pesadas de entornos monolíticos como GNOME o KDE; y

descansa en el motor Scintilla (`ScintillaGTK`) para administrar la representación del búfer de texto, gestionando el resaltado sintáctico mediante analizadores léxicos especializados (*lexers*), el plegado jerárquico de bloques de código (*code folding*), márgenes e indicadores de sangría.

A diferencia de entornos modernos basados en máquinas virtuales intermedias (como Eclipse) o marcos web empaquetados en navegadores (como Electron), Geany compila directamente a binario ejecutable ELF nativo en C/C++, alcanzando tiempos de arranque prácticamente instantáneos y un consumo base de memoria RAM inferior a 50 megabytes. Entre sus capacidades técnicas sobresale el soporte para más de 50 lenguajes de programación (C, C++, Python, Java, PHP, HTML, Rust), la integración de `ctags` para generar un árbol de símbolos en tiempo real que indexa funciones, clases y variables, un sistema de construcción configurable que invoca directamente compiladores nativos (`gcc`, `make`, depuradores o intérpretes), y una API en C (`geanyplugin.h`) para complementos compartidos. Su propósito fundamental es ofrecer una estación de desarrollo ágil y eficiente, idónea para sistemas con recursos de hardware restringidos, servidores remotos con escritorios ligeros o desarrolladores que demandan inmediatez sin penalizaciones de latencia.

6.3. Capas de compatibilidad y traducción de API: Wine

Wine es una capa de compatibilidad de código abierto diseñada para permitir la ejecución de aplicaciones y binarios compilados para Microsoft Windows en sistemas operativos tipo UNIX, tales como GNU/Linux, FreeBSD y macOS (WineHQ, 2024). Su acrónimo recursivo (*Wine Is Not an Emulator*) explicita su principio arquitectónico medular: Wine no es un emulador de hardware ni un hipervisor de virtualización; las instrucciones de código máquina del binario se ejecutan directamente sobre la CPU física del equipo anfitrión a velocidad nativa. Su arquitectura opera mediante la reimplementación limpia en espacio de usuario del conjunto de bibliotecas de vínculos dinámicos (DLL, *Dynamic Link Libraries*) que conforman las API Win32 y Win64. Cuando una aplicación en formato ejecutable portable (PE, *Portable Executable*) invoca una función de Windows, la llamada es interceptada por los módulos equivalentes de Wine y traducida en llamadas al sistema nativas POSIX o servicios de subsistemas UNIX.

Esta traducción abarca tres frentes cardinales: el mapeo de primitivas del núcleo de Windows (`ntdll.dll` y `kernel32.dll`) a llamadas POSIX como `mmap`, `clone`, `fork` y descriptores de archivo; la traducción del subsistema gráfico y de ventanas (`user32.dll` y `gdi32.dll`) hacia los protocolos X11 o Wayland; y la aceleración multimedia, que traduce llamadas Direct3D hacia las API estándar OpenGL y Vulkan mediante motores especializados (WineD3D, DXVK y vkd3d), canalizando el audio hacia servidores como PulseAudio o PipeWire. El estado global y la sincronización entre procesos descansan en el demonio coordinador `wineserver`, el cual implementa el espacio de nombres de manejadores (*handles*), administra primitivas de sincronización (semáforos, eventos, exclusiones mutuas), gestiona la comunicación IPC y mantiene la estructura del Registro de Windows. Cada entorno opera dentro de un directorio aislado denominado prefijo Wine (`WINEPREFIX`), recreando la estructura de discos y archivos de configuración de Windows. Su propósito es garantizar la interoperabilidad de software empresarial, herramientas de ingeniería y aplicaciones heredadas de Windows sobre infraestructuras libres tipo UNIX sin requerir licencias de Microsoft ni degradar el rendimiento con emulación.

6.4. Matriz comparativa de herramientas de software

Para sintetizar las divergencias operativas y la función técnica de estas tres plataformas, en la Tabla 6 se contrastan su categoría de diseño, sus mecanismos de interacción con el sistema operativo y su utilidad en la infraestructura informática.

Tabla 6: Comparación técnica de herramientas de entorno gráfico, desarrollo y compatibilidad.

Herramienta	Categoría y base técnica	Mecanismo de interacción con el SO	Función cardinal y caso de uso
Compiz	Gestor de composición de ventanas basado en OpenGL sobre X11.	Redirección de pixmaps fuera de pantalla y enlace directo a texturas en la GPU vía GLX.	Aceleración gráfica del escritorio, efectos 3D, gestión de espacios de trabajo y ergonomía visual.
Geany	Entorno de desarrollo integrado (IDE) ligero en C/C++ (GTK+ y Scintilla).	Ejecución nativa sin capas intermedias; invoca cadenas de construcción (`gcc`, `make`) del sistema.	Edición y compilación ágil de código fuente en más de 50 lenguajes con mínimo consumo de RAM ($<$50 MB).
Wine	Capa de compatibilidad de API Win32/Win64 para sistemas tipo UNIX.	Reimplementación de bibliotecas DLL en espacio de usuario, traducción a llamadas POSIX y demonio `wineserver`.	Ejecución nativa de binarios de Windows (.exe/.dll) en Linux/BSD sin emulación de hardware ni hipervisores.

7. Conclusiones personales

La exploración de los sistemas operativos distribuidos y de red demuestra que la computación contemporánea ha superado definitivamente el paradigma del procesamiento centralizado. La aspiración teórica del sistema operativo distribuido puro (DO/S), orientado a proyectar una imagen única de sistema con transparencia total de acceso, ubicación, migración y concurrencia según el modelo RM-ODP, constituye uno de los ideales más ambiciosos de las ciencias de la computación. No obstante, sus compromisos de diseño explican por qué la industria adoptó prevalentemente los sistemas operativos de red (NOS) y las arquitecturas cliente-servidor: la latencia física de interconexión, las fallas parciales y los límites del teorema CAP restringen la sincronización estricta y la memoria compartida distribuida. Por ende, el modelo de red pragmático, donde cada nodo preserva su autonomía y colabora mediante protocolos abiertos, se consolidó como el estándar global.

El análisis de los modelos de comunicación revela una transición continua desde sockets de bajo nivel hacia llamadas a procedimientos remotos (RPC), métodos distribuidos (RMI) y servicios desacoplados. En este contexto, un servidor contemporáneo no se define por un chasis físico exclusivo, sino por la especialización de su sistema operativo. Los núcleos para servidor reorientan sus prioridades internas: postergan la latencia interactiva para maximizar el rendimiento global de la CPU, ajustan la tasa de intercambio de memoria para blindar la caché de páginas en RAM, aplican escalado lateral de recepción en interfaces

de red y emplean multiplexación asíncrona no bloqueante de descriptores de entrada/salida para gestionar decenas de miles de conexiones concurrentes con mínima sobrecarga.

La comparativa de las diez plataformas de servidor constata la madurez y diversificación del ecosistema operativo. No existe una solución universal, sino respuestas de ingeniería adaptadas a nichos específicos. Microsoft Windows Server lidera la integración corporativa mediante Active Directory y directivas unificadas. La familia GNU/Linux domina los centros de datos y la nube, equilibrando distribuciones comerciales de misión crítica con confinamiento mandatorio mediante SELinux (RHEL, SLES) y plataformas comunitarias de estabilidad rigurosa (Debian, Rocky Linux, Ubuntu Server). A su vez, los derivados de UNIX revalidan su excelencia: FreeBSD con el rendimiento de su pila de red y almacenamiento OpenZFS de 128 bits; OpenBSD con su paradigma de seguridad proactiva y mitigaciones en memoria; y Solaris/illumos con el legado pionero de DTrace y zonas de aislamiento. Por último, hipervisores bare-metal como VMware ESXi confirman que la virtualización en anillo 0 maximiza la densidad y eficiencia energética del hardware.

En conectividad y seguridad perimetral, los mecanismos basales en capas 3 y 4 sustentan la viabilidad de la red. La traducción NAT y el protocolo DHCP no solo mitigaron el agotamiento del direccionamiento público IPv4, sino que automatizaron la configuración dinámica de terminales sin fricción administrativa. Simultáneamente, los cortafuegos con inspección de estados en espacio de núcleo (netfilter, PF) garantizan filtrado perimetral con mínima sobrecarga de interrupciones. Esta defensa se complementa con servidores proxy que desacoplan el tráfico entrante, absorben ataques de denegación de servicio, efectúan terminación SSL/TLS para descargar de cómputo criptográfico al backend y aceleran la entrega de recursos mediante memoria intermedia de caché.

La integración de servicios corporativos resuelve la complejidad operativa organizacional. El modelo AAA mediante RADIUS centraliza la autenticación y políticas de acceso en redes cableadas e inalámbricas. En compartición de datos, Samba hace converger entornos heterogéneos al mapear permisos POSIX sobre listas ACL de Windows con total transparencia. Asimismo, la estandarización documental de CUPS vía IPP y la evolución del acceso remoto hacia túneles criptográficos modernos como WireGuard y OpenVPN ratifican el equilibrio entre confidencialidad, robustez y desempeño. Por su parte, la pila LAMP ejemplifica la vigencia del desacoplamiento modular abierto, donde el sistema operativo, el servidor web, el motor transaccional ACID y el intérprete de aplicación interactúan mediante sockets locales con extraordinaria eficiencia.

El análisis de herramientas de entorno gráfico, desarrollo y capas de compatibilidad vincula al sistema operativo con la productividad del usuario técnico. Compiz demostró que la aceleración gráfica por hardware y la composición tridimensional desacoplada del servidor X11 mejoran la ergonomía visual y la gestión de espacios de trabajo sin penalizar a la CPU. En paralelo, entornos como Geany evidencian que un IDE ágil y basado en binarios nativos C/C++ puede brindar capacidades completas de programación con un consumo basal inferior a 50 MB de RAM, desafiando la sobrecarga de herramientas sobredimensionadas. En este sentido, Wine ratifica que la interoperabilidad binaria no requiere la virtualización costosa del procesador, sino la traducción limpia y precisa de llamadas a la API de Windows a primitivas POSIX en tiempo de ejecución.

En conclusión, el estudio de los sistemas distribuidos y orientados a servidores exige entender la computación como una disciplina sistémica donde hardware, núcleo y red interactúan con rigor matemático y algorítmico. Frente a la expansión de arquitecturas nativas de nube, clústeres masivos y microservicios, el dominio de los fundamentos analizados —arbitraje de concurrencia, aislamiento de memoria, seguimiento de conexiones e inter-

operabilidad de protocolos— se erige como una competencia cardinal para el ingeniero en computación, facultándolo para diseñar infraestructuras distribuidas escalables, seguras y resilientes ante los retos tecnológicos contemporáneos.

Referencias

Canonical Ltd. (2024). *Ubuntu server guide: System architecture, administration and storage*. Consultado el 1 de noviembre de 2026, desde https://ubuntu.com/server/docs

Deitel, H. M., Deitel, P. J., & Choffnes, D. R. (2004). *Operating systems* (3.ª ed.). Pearson Prentice Hall.

Droms, R. (1997). *Dynamic Host Configuration Protocol* (inf. téc. N.º RFC 2131). IETF. Consultado el 4 de noviembre de 2026, desde https://www.rfc-editor.org/rfc/rfc2131

Freedesktop.org. (2023). *Compiz technical overview: Compositing window management on X11*. Consultado el 4 de noviembre de 2026, desde https://www.freedesktop.org/wiki/Software/CompizTechOverview/

McHoes, A. M., & Flynn, I. M. (2011). *Sistemas operativos* (6.ª ed.). Cengage Learning Editores.

Microsoft Corporation. (2024). *Windows server documentation and architecture overview*. Consultado el 4 de noviembre de 2026, desde https://learn.microsoft.com/en-us/windows-server/

Oracle Corporation. (2023). *Oracle solaris 11.4 system administration: Common systems architecture and zfs*. Consultado el 4 de noviembre de 2026, desde https://docs.oracle.com/cd/E37838_01/

Patel, B., Aboba, B., Dixon, W., Zorn, G., & Booth, S. (2001). *Securing L2TP using IPsec* (inf. téc. N.º RFC 3193). IETF. Consultado el 4 de noviembre de 2026, desde https://www.rfc-editor.org/rfc/rfc3193

Red Hat, Inc. (2024). *Red hat enterprise linux 9: System design and architecture overview*. Consultado el 4 de noviembre de 2026, desde https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/9

Rigney, C., Willens, S., Rubens, A., & Simpson, W. (2000). *Remote Authentication Dial In User Service (RADIUS)* (inf. téc. N.º RFC 2865). IETF. Consultado el 4 de noviembre de 2026, desde https://www.rfc-editor.org/rfc/rfc2865

Rocky Enterprise Software Foundation. (2024). *Rocky linux documentation: Enterprise server deployment guide*. Consultado el 4 de noviembre de 2026, desde https://docs.rockylinux.org/

Silberschatz, A., Galvin, P. B., & Gagne, G. (2006). *Fundamentos de sistemas operativos* (7.ª ed.). McGraw-Hill/Interamericana de España.

Srisuresh, P., & Egevang, K. (2001). *Traditional IP Network Address Translator (Traditional NAT)* (inf. téc. N.º RFC 3022). IETF. Consultado el 2 de noviembre de 2026, desde https://www.rfc-editor.org/rfc/rfc3022

Stallings, W. (2018). *Operating systems: Internals and design principles* (9.ª ed.) [Global Edition]. Pearson Education Limited.

SUSE LLC. (2023). *Suse linux enterprise server 15 sp5: Administration and architecture guide*. Consultado el 4 de noviembre de 2026, desde https://documentation.suse.com/sles/15-SP5/

The Apache Software Foundation. (2024). *Apache HTTP server architecture: Multi-processing modules (MPMs)*. Consultado el 4 de noviembre de 2026, desde https://httpd.apache.org/docs/2.4/mpm.html

The Debian Project. (2023). *Debian gnu/linux 12 (bookworm) release notes*. Consultado el 25 de octubre de 2026, desde https://www.debian.org/releases/bookworm/releasenotes

The FreeBSD Project. (2023). *Freebsd 14.0-release release notes*. Consultado el 25 de octubre de 2026, desde https://www.freebsd.org/releases/14.0R/relnotes/

The Geany Project. (2024). *Geany: Fast and lightweight IDE architecture and documentation*. Consultado el 4 de noviembre de 2026, desde https://www.geany.org/documentation/

The OpenBSD Project. (2024). *Openbsd 7.5: Architecture, security mitigations and system internals*. Consultado el 4 de noviembre de 2026, desde https://www.openbsd.org/faq/

The OpenPrinting Project. (2024). *CUPS architecture: Internet printing protocol and spooler design*. Consultado el 4 de noviembre de 2026, desde https://openprinting.github.io/cups/doc/overview.html

The Samba Team. (2024). *Samba: Integrated SMB/CIFS services and active directory domain controller guide*. Consultado el 4 de noviembre de 2026, desde https://www.samba.org/samba/docs/

VMware, Inc. (2023). *Vmware esxi architecture: Type-1 hypervisor overview and vmkernel internals*. Consultado el 4 de noviembre de 2026, desde https://docs.vmware.com/en/VMware-vSphere/8.0/vsphere-esxi-installation/GUID-62CE9B97-84C1-4D04-8025-E02CB4AC4B82.html

WineHQ. (2024). *Wine developer's guide: Architecture and win32 subsystem internals*. Consultado el 4 de noviembre de 2026, desde https://wiki.winehq.org/Wine_Developer%27s_Guide/Architecture